Node.js Exam Preparation Guide

Section 1: MCQs (One-Liner Answers)

1. Role of the script field in `package.json`: It defines custom aliases for shell commands (e.g., `npm start` to run the server), allowing you to automate tasks like testing, building, or starting the application.

2. How to perform successful verification of Node.js installation on Windows: Open the command prompt (terminal) and run the command `node -v` or `node --version`; if a version number is returned, the installation is verified.

3. Name the core module in Node.js: Common core modules include `fs` (file system), `http`, `path`, and `os`. (Note: Your notes specifically point to `fs`).

4. Mechanism Node handles multiple client requests efficiently: It uses the **Event Loop** combined with a single-threaded, event-driven, non-blocking I/O architecture.

5. Which is NOT the role of `package.json`: It does **not** store the actual "source code" of the application; it only stores metadata and dependency lists.

6. How to exit from the REPL session properly: Press `Ctrl + C` twice, or type `.exit` and press Enter.

7. Default buffer size in Node.js: Streams often default to a highWaterMark of **64KB** (65536 bytes) for reading files.

8. Which property is true for a local module: It is **user-created** and must be imported using a relative path (e.g., `require('./mymodule')`).

9. Command to install a package globally: `npm install -g <package_name>` (e.g., `npm install -g nodemon`).

10. Procedure to create a Node.js project: Run `npm init` in the terminal and answer the prompts to generate the `package.json` file.

11. What happens during package installation if there is no `-g`: The package is installed **locally** in the current project's `node_modules` folder, not system-wide.

12. The property that does not match with Buffer: The statement "they require import" is technically false/mismatched because the `Buffer` class is a global object in Node.js and does not require an explicit `require()` statement.

13. Command to convert a task to update an NPM package: `npm update <package_name>` (or simply `npm update` to update all packages).

14. Identify the file that represents project meta-data: `package.json`.

15. Which dependency is used for production: Packages listed under `"dependencies"` (as opposed to `"devDependencies"`).

Section 2: Module 1 / IAQ (3 Marks Each)

1. Describe how Node.js achieves non-blocking I/O and state its benefit.

- **Mechanism:** Node.js uses a single thread. When an I/O operation (like reading a file) is requested, Node sends it to the system kernel (via libuv) and continues executing the next line of code immediately without waiting.
- **Callback:** When the operation finishes, the kernel signals Node.js, and the callback function associated with that task is added to the Event Queue to be executed.
- **Benefit:** It allows a single server to handle thousands of concurrent connections efficiently without consuming memory for new threads for every request.

2. Working mechanism of V8 engine to help Javascript execution in Node.js.

- **Compilation:** V8 (written in C++) takes JavaScript code and compiles it directly into native machine code (JIT - Just In Time compilation) rather than interpreting it line-by-line.
- **Memory Management:** It manages the **Memory Heap** (where memory allocation happens) and the **Call Stack** (where stack frames are executed).
- **Optimization:** V8 uses "hidden classes" and "inline caching" to optimize property access and execution speed, making JS run fast enough for server-side tasks.

3. How errors are handled in REPL and why it is critical in Node.js.

- **REPL Handling:** In the REPL, syntax errors are caught and printed immediately. If a runtime error occurs, the REPL prints the stack trace but usually keeps the session alive for you to try again.
- **Critical Nature:** In a running Node.js application, error handling is critical because Node is **single-threaded**.
- **Consequence:** If an "Uncaught Exception" occurs, the entire process crashes. Unlike multi-threaded servers where only one thread dies, in Node.js, the whole server goes down, affecting all users.

4. Can you install multiple versions of Node.js on a system?

- **Directly:** No, installing a new version typically overwrites the old one in the system path.
- **Using NVM:** Yes, you can use a tool called **NVM (Node Version Manager)**.
- **Benefit:** NVM allows you to switch between versions (e.g., `nvm use 14.0.0` or `nvm use 18.0.0`) instantly, which is useful for testing projects that require different legacy versions.

5. Explain worker threads in Node.js.

- **Purpose:** While Node is single-threaded, the `worker_threads` module allows execution of JavaScript in parallel threads.
- **Use Case:** They are specifically designed for **CPU-intensive tasks** (like image processing or cryptography) which would otherwise block the main Event Loop.

- **Structure:** Unlike child processes, Worker Threads share memory (via `SharedArrayBuffer`), making them more lightweight and faster for passing data.

6. Comparison with traditional web servers vs Node.js Event Loop.

- **Traditional (e.g., Apache):** Uses a "Thread-based" model. It creates a new thread for every incoming client request. High memory usage per user.
- **Node.js:** Uses a "Single-threaded Event Loop." One thread handles all connections.
- **Performance:** Node.js performs significantly better for **I/O-bound applications** (Data streaming, Real-time chats).
- **Challenge:** Node.js struggles with **CPU-bound** tasks because if the single thread is busy calculating, it cannot handle new incoming requests (blocking the loop).

7. Create and call a function in Javascript with proper example.

- **Definition:** A function is a reusable block of code.
- **Code Example:**

```
// 1. Function Declaration
function addNumbers(a, b) {
    return a + b;
}

// 2. Calling the function
const result = addNumbers(5, 10);
console.log("The sum is:", result); // Output: The sum is: 15
```

8. Identify the step to check the version of Node.js and NPM install on your system.

- **Step 1:** Open your Command Prompt (Windows) or Terminal (Mac/Linux).
- **Step 2 (Node):** Type `node -v` and press Enter. It will output something like `v18.16.0`.
- **Step 3 (NPM):** Type `npm -v` and press Enter. It will output the version of the Node Package Manager, e.g., `9.5.1`.

9. How can you check the installed version of a Node.js package?

- **Local Package:** Run `npm list <package-name>` in the terminal inside your project folder.
- **Global Package:** Run `npm list -g <package-name>`.
- **Metadata:** You can also manually open the `package.json` file and look under the `dependencies` object to see the semantic version range (e.g., `"express": "^4.18.2"`).

10. Compare the effect of synchronous and asynchronous file system operation in a high traffic situation.

- **Synchronous (`fs.readFileSync`):** This **blocks** the main thread. If 1000 users request a file, the server handles user 1, makes user 2-1000 wait until user 1 is done. This kills performance in high traffic.

- **Asynchronous (`fs.readFile`):** This is **non-blocking**. The server initiates the read for user 1 and immediately moves to handle user 2.
- **Result:** Asynchronous operations allow a Node.js server to handle thousands of requests concurrently, making it suitable for high-traffic environments.

11. Effect of synchronous & asynchronous operation on the performance of Node.js server. *(Similar to Q10, but focused on server health)*

- **Synchronous:** Degrades performance. The Event Loop halts. The server appears "frozen" to other clients while the operation completes.
- **Asynchronous:** Maximizes performance. The CPU stays busy handling requests while the I/O subsystem handles the file/network work. The Event Loop keeps spinning, ensuring low latency.

Section 3: CT-1 SAQ (3 Marks Each)

1. Describe some methods of the OS module with examples.

- `os.platform()` : Returns the operating system platform (e.g., `'win32'` , `'linux'`).
- `os.cpus()` : Returns an array of objects containing information about each logical CPU core.
- `os.freemem()` : Returns the amount of free system memory in bytes.
- `os.userInfo()` : Returns information about the currently effective user (username, homedir).

2. How can you define Node.js?

- **Definition:** Node.js is an open-source, cross-platform JavaScript **runtime environment** that executes JavaScript code outside a web browser.
- **Core Tech:** It is built on Chrome's **V8 JavaScript engine**.
- **Architecture:** It uses an **event-driven, non-blocking I/O model** that makes it lightweight and efficient for building scalable network applications.

3. Construct and compare Node.js with a traditional web server.

- **Architecture:** Node.js is Single-Threaded Event Driven. Traditional servers (Apache/Tomcat) are Multi-Threaded/Blocking.
- **Memory:** Node.js uses constant memory (one thread). Traditional servers require allocating RAM for every new client thread, leading to scaling limits.
- **Use Case:** Node is best for real-time apps (Chat, Streaming). Traditional is often better for heavy computation or strictly synchronous logic.

4. What is the implementation of Node in the real world?

- **Streaming Services: Netflix** uses Node.js to streamline the user interface and reduce startup time for streams.
- **Real-time Collaboration: Trello** and Google Docs-style apps use Node for instant updates across users via WebSockets.

- **Transportation: Uber** uses Node.js for its massive reliable matching system because Node handles asynchronous data (drivers/riders moving) efficiently.

5. Compare between Buffer and Array in Node.

- **Content:** Arrays can hold any data type (strings, numbers, objects). Buffers are specifically designed to hold **raw binary data**.
- **Memory:** Arrays are allocated in the V8 heap and can resize. Buffers correspond to fixed-size raw memory locations outside the V8 heap.
- **Usage:** Arrays are for general logic. Buffers are for processing streams, files, and TCP packets.

6. How you can set up a basic development environment in Node.js.

1. **Install:** Download the LTS version from nodejs.org and install it. Verify with `node -v`.
2. **Editor:** Install a code editor (VS Code is standard).
3. **Initialize:** Create a folder, open terminal, run `npm init -y` to create `package.json`.
4. **Run:** Create an `index.js` file and run it using `node index.js`.

7. Advantage & disadvantage of REPL.

- **Advantage:** Great for **quick experimentation**. You can test a function, regex, or math calculation immediately without creating a file. Good for debugging simple logic.
- **Disadvantage:** Not suitable for writing complex applications. Code is **lost** once you exit the session (unless you save explicitly). It lacks the structure of a proper IDE.

8. How NPM works in Node.js?

- **Repository:** There is a massive cloud database (registry) containing published JS packages.
- **CLI:** The `npm` command-line tool allows you to fetch these packages.
- **Manifest:** When you run `npm install`, it reads/writes to `package.json` and places the actual code into a `node_modules` folder, allowing your app to `require()` that functionality.

9. Define core, local, and third-party modules of Node.js with example.

- **Core Modules:** Built into Node.js, no installation needed. *Example:* `http`, `fs`, `events`.
- **Local Modules:** Created by the user in the project. Imported via file path. *Example:* `const myLog = require('./log.js')`.
- **Third-Party Modules:** Installed via NPM, created by the community. *Example:* `express`, `mongoose`, `nodemon`.

10. Analyse the impact of synchronous file operation in a high traffic Node.js server.
(Refer to Section 2, Q10).

- **Impact:** It creates a **bottleneck**. Since Node allows only one operation at a time on the main thread, a synchronous file read halts the entire server.

- **Traffic Consequence:** In high traffic, requests pile up (Time-to-First-Byte increases drastically), eventually causing the server to time out or crash due to queue overflow.

11. Describe how can you install NPM package locally and globally.

- **Locally:** Run `npm install <package-name>` . This puts the package in the current project's `node_modules` . Used for project-specific dependencies (e.g., Express).
- **Globally:** Run `npm install -g <package-name>` . This puts the package in the system's root directory. Used for CLI tools you run from anywhere (e.g., Nodemon, Angular CLI).

12. Why buffer is important to handle data? How can you implement it?

- **Importance:** JavaScript traditionally didn't handle binary data well (it handled strings). Buffers are essential for reading images, file streams, or network data packets efficiently without converting them to strings.
- **Implementation:**

```
// Creating a buffer from a string
const buf = Buffer.from('Hello');

// Allocating a specific size (e.g., 10 bytes)
const buf2 = Buffer.alloc(10);

console.log(buf.toString()); // Converts binary back to 'Hello'
```

13. Explain purpose of URL module in Node.js.

- **Purpose:** To parse, construct, and manipulate URLs easily. It splits a web address into readable parts (Host, Path, Query Parameters).
- **Usage:** It helps extract specific information from a request URL.
- **Example:**

```
const url = require('url');
const myUrl = new URL('[https://example.com/page?id=5](https://example.com/pa
console.log(myUrl.searchParams.get('id')); // Outputs: 5
```